

《人工智能基础 A》实验报告 5

语义相似度与小说分析

学号

姓名：董卓然

一、实验报告（实验文档中只包含此部分即可，将截图放在此处）

1. 报告内容应完整覆盖实验要求，格式规范、排版整洁【占 10 分】。
2. 完成相似度计算【占 15 分】

苹果的向量
(0.96, 0.77, 0.85, 0.15)

大米的向量
(0.18, 0.22, 0.05, 0.93)

$A \cdot B = 0.5242$

$|A| = \sqrt{0.96^2 + 0.77^2 + 0.85^2 + 0.15^2} \approx 1.5032$

$|B| = \sqrt{0.18^2 + 0.22^2 + 0.05^2 + 0.93^2} \approx 0.9738$

$\cos \theta = \frac{0.5242}{1.5032 \times 0.9738} \approx 0.358$

此为余弦相似度

定义 Jaccard 相似度:

分子 = $\sum \min(a_i, b_i) = 0.18 + 0.22 + 0.05 + 0.15 = 0.60$

分母 = $\sum \max(a_i, b_i) = 0.96 + 0.77 + 0.85 + 0.93 = 3.51$

$J = \frac{0.60}{3.51} \approx 0.1709$

3250102672
D2R

3. 词袋模型实践【占 15 分】

3.1. 参考上述代码, 尝试构建两句话以上的词袋模型并计算相似度

```
import jieba
jieba.load_userdict('data/names_xiyou.txt')
jieba.load_userdict('data/names_sanguo.txt')
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 2.1 两句话
sentence1 = "孙悟空保护唐僧一路西行取经"
sentence2 = "悟空护送师父前往西天求真经"

@v
def jieba_cut(sentence):
    return " ".join(jieba.cut(sentence))

s1_cut = jieba_cut(sentence1)
s2_cut = jieba_cut(sentence2)
print("分词1:", s1_cut)
print("分词2:", s2_cut)

vectorizer = CountVectorizer()
X = vectorizer.fit_transform([s1_cut, s2_cut])
sim = cosine_similarity(X[0:1], X[1:2])
print(f"\n[2.1] 两句话余弦相似度: {sim[0][0]:.4f}")

# 2.2 多句话
sentences_bow = [
    "刘备是蜀汉的开国皇帝",
    "刘备建立了蜀国成为一代君主",
    "曹操是魏国的奠基人",
    "诸葛亮辅佐刘备治理蜀汉",
    "孙权统治东吴多年",
]
tokenized_bow = [jieba_cut(s) for s in sentences_bow]

vec2 = CountVectorizer()
X2 = vec2.fit_transform(tokenized_bow)
sim_matrix = cosine_similarity(X2)

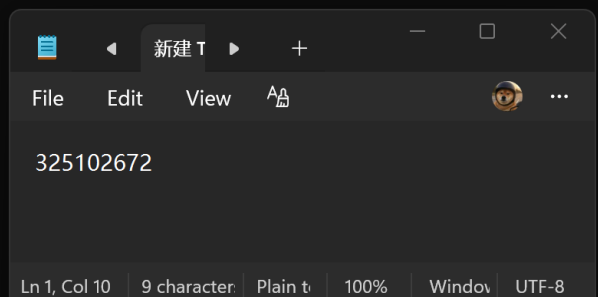
print("\n[2.2] 多句话余弦相似度矩阵:")
for i, row in enumerate(sim_matrix):
    print(f"句{i+1}:", " ".join([f"{v:.2f}" for v in row]))
```

```
(nlp311) C:\Users\lenovo\Desktop\实验5>code .

(nlp311) C:\Users\lenovo\Desktop\实验5>python lab5.py
Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\lenovo\AppData\Local\Temp\jieba.cache
Loading model cost 0.402 seconds.
Prefix dict has been built successfully.
分词1: 孙悟空 保护 唐僧 一路 西行 取经
分词2: 悟空 护送 师父 前往 西天 求真经

[2.1] 两句话余弦相似度: 0.0000

[2.2] 多句话余弦相似度矩阵:
句1: 1.00 0.20 0.00 0.45 0.00
句2: 0.20 1.00 0.00 0.18 0.00
句3: 0.00 0.00 1.00 0.00 0.00
句4: 0.45 0.18 0.00 1.00 0.00
句5: 0.00 0.00 0.00 0.00 1.00
```



分词1: 孙悟空 保护 唐僧 路 西行 取经
分词2: 悟空 护送 师父 前往 西天 求取 真经

[2.1] 两句话余弦相似度: 0.0000

[2.2] 多句话余弦相似度矩阵:

句1: 1.00 0.20 0.00 0.45 0.00
句2: 0.20 1.00 0.00 0.18 0.00
句3: 0.00 0.00 1.00 0.00 0.00
句4: 0.45 0.18 0.00 1.00 0.00
句5: 0.00 0.00 0.00 0.00 1.00

我喜欢在清晨喝一杯热茶，享受安静的时光。
今天是我生日，我准备和家人一起庆祝。
科技改变了我们的生活，尤其是人工智能的发展。
早晨的阳光洒在大地上，给一切带来了温暖。
我喜欢在每天下午来一杯咖啡，它能让我更加集中精神。
每当下雨时，我喜欢在窗前静静地看着雨滴落下。
昨晚我和朋友一起去看了一场电影，感觉很放松。
电脑和手机已经成为现代社会不可或缺的一部分。
他每天都在健身房锻炼，已经养成了好习惯。
今天是我的生日，但我与朋友约好了一起在海底捞庆祝。
我喜欢读科幻小说，因为它们带我进入了另一个未知的世界。

热图已保存

4. 使用 Word2Vec 计算文本相似度【占 15 分】



```

import numpy as np
from gensim.models import Word2Vec
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

sentences_w2v = [
    "我喜欢在清晨喝一杯热茶，享受安静的时光。",
    "今天是我生日，我准备和家人一起庆祝。",
    "科技改变了我们的生活，尤其是人工智能的发展。",
    "早晨的阳光洒在大地上，给一切带来了温暖。",
    "我喜欢在每天下午来一杯咖啡，它能让我更加集中精神。",
    "每当下雨时，我喜欢在窗前静静地看着雨滴落下。",
    "昨晚我和朋友一起去看了一场电影，感觉很放松。",
    "电脑和手机已经成为现代社会不可或缺的一部分。",
    "他每天都在健身房锻炼，已经养成了好习惯。",
    "今天是我的生日，但我与朋友约好了一起在海底捞庆祝。",
    "我喜欢读科幻小说，因为它们带我进入了另一个未知的世界。"
]

tokenized = [list(jieba.cut(s)) for s in sentences_w2v]
for s in tokenized:
    print(" ".join(s))

model_w2v = Word2Vec(tokenized, vector_size=100, window=5,
                      min_count=1, sg=0, epochs=10)

def sentence_vector(tokens):
    vecs = [model_w2v.wv[w] for w in tokens if w in model_w2v.wv]
    return np.mean(vecs, axis=0) if vecs else np.zeros(100)

sent_vecs = np.array([sentence_vector(s) for s in tokenized])
cosine_sim = cosine_similarity(sent_vecs)

plt.figure(figsize=(10, 8))
labels = [f"句子{i+1}" for i in range(len(sentences_w2v))]
sns.heatmap(cosine_sim, annot=True, cmap="coolwarm",
            xticklabels=labels, yticklabels=labels, fmt=".2f")
plt.title("句子余弦相似度热图")
plt.tight_layout()
plt.savefig("result/task3_heatmap.png", dpi=150)
plt.show()
print("热图已保存")

```



5. 小说分析【占 45 分】

5.1. 参考示例，西游记的人物分析【15】

```
def cutWordsToTrain(filename):
    words = []
    with open(filename, 'rb') as f:
        raw = f.read()
    try:
        text = raw.decode('utf-8')
    except:
        text = raw.decode('gbk', errors='ignore')

    stopwords = set()
    with open('data/stopwords_hit.txt', 'r', encoding='utf-8') as f:
        for w in f:
            stopwords.add(w.strip())

    for line in text.split('\n'):
        line = line.strip()
        if not line:
            continue
        result = list(jieba.cut(line, cut_all=False))
        result = [x for x in result if x.isalnum()]
        result = [x.lower() for x in result]
        result = [x for x in result if x not in stopwords]
        if result:
            words.append(result)
    return words

print("正在分词西游记，请稍候...")
words_xy = cutWordsToTrain('data/西游记.txt')
print(f"分词完成，共{len(words_xy)}行")

model_xy = Word2Vec(words_xy, hs=0, min_count=2, window=5,
                    vector_size=100, epochs=10, sg=0, compute_loss=True)
model_xy.save('result/xiyou.model')
print("模型训练完成")

print("\n与唐僧关系最密切的5个词:")
for word, score in model_xy.wv.most_similar('唐僧', topn=5):
    print(f" {word}: {score:.4f}")

print(f"\n唐僧 ↔ 孙悟空 亲密度: {model_xy.wv.similarity('唐僧', '孙悟空'):.4f}")
print(f"孙悟空 ↔ 猪八戒 亲密度: {model_xy.wv.similarity('孙悟空', '猪八戒'):.4f}")

print("\n唐僧的词嵌入(前10维):", model_xy.wv['唐僧'][:10])
print("孙悟空的词嵌入(前10维):", model_xy.wv['孙悟空'][:10])
```

我喜欢读科幻小说，因为它们带我进入了一个未知的世界。

热图已保存

正在分词西游记，请稍候...

分词完成，共3416行

模型训练完成

与唐僧关系最密切的5个词:

题目: 0.8782

少待: 0.8753

长老: 0.8708

一听: 0.8636

搀住: 0.8614

唐僧 ↔ 孙悟空 亲密度: 0.7923

孙悟空 ↔ 猪八戒 亲密度: 0.9298

唐僧的词嵌入(前10维): [-0.6108122 0.43187544 0.07800721 -0.29312402 0.059432 -1.7439393
-0.0859475 0.7675674 -1.2649684 -0.79857063]

孙悟空的词嵌入(前10维): [-0.0864807 -0.01102244 -0.14346056 -0.07250317 0.3268078 -1.4847214
0.5775575 1.2373077 -0.40258595 -0.36781836]

正在分词三国演义，请稍候...

5.2. 根据上述代码，完成下列分析【30】

运用自然语言处理技术识别与分析《三国演义》：分析刘备、张飞、关羽、曹操、孙权、诸葛亮等人物的关系密切度

```
jieba.load_userdict('data/names_xiyou.txt') if os.path.exists('data/names_xiyou.txt') else None

def cutWordsToTrain(filename):
    words = []
    with open(filename, 'rb') as f:
        raw = f.read()
    try:
        text = raw.decode('utf-8')
    except:
        text = raw.decode('gbk', errors='ignore')

    stopwords = set()
    with open('data/stopwords_hit.txt', 'r', encoding='utf-8') as f:
        for w in f:
            stopwords.add(w.strip())

    for line in text.split('\n'):
        line = line.strip()
        if not line:
            continue
        result = list(jieba.cut(line, cut_all=False))
        result = [x for x in result if x.isalnum()]
        result = [x.lower() for x in result]
        result = [x for x in result if x not in stopwords]
        if result:
            words.append(result)
    return words

print("正在分词西游记，请稍候...")
words_xy = cutWordsToTrain('data/西游记.txt')
print(f"分词完成，共{len(words_xy)}行")

model_xy = Word2Vec(words_xy, hs=0, min_count=2, window=5,
                    vector_size=100, epochs=10, sg=0, compute_loss=True)
model_xy.save('result/xiyou.model')
print("模型训练完成")

print("\n\n与唐僧关系最密切的5个词：")
for word, score in model_xy.wv.most_similar('唐僧', topn=5):
    print(f" {word}: {score:.4f}")

print(f"\n\n唐僧 + 孙悟空 亲密度: {model_xy.wv.similarity('唐僧', '孙悟空'):.4f}")
print(f"\n\n唐僧 + 猪八戒 亲密度: {model_xy.wv.similarity('唐僧', '猪八戒'):.4f}")

print("\n\n唐僧的词嵌入[前10维]:", model_xy.wv['唐僧'][:10])
print("\n\n孙悟空的词嵌入[前10维]:", model_xy.wv['孙悟空'][:10])

# =====
# 任务4.2: 三国演义人物关系分析
# =====

print("\n\n正在分词三国演义，请稍候...")
words_sg = cutWordsToTrain('data/三国演义.txt')
print(f"分词完成，共{len(words_sg)}行")

model_sg = Word2Vec(words_sg, hs=0, min_count=2, window=5,
                    vector_size=100, epochs=10, sg=0, compute_loss=True)
model_sg.save('result/sanguo.model')
print("模型训练完成")

characters = ['刘备', '张飞', '关羽', '曹操', '孙权', '诸葛亮']

for char in characters:
    print(f"\n\n与【{char}】最相关的5个词：")
    for word, score in model_sg.wv.most_similar(char, topn=5):
        print(f" {word}: {score:.4f}")

print("\n\n六人两两亲密度：")
for a in characters:
    for b in characters:
        if a < b:
            s = model_sg.wv.similarity(a, b)
            print(f" {a} * {b}: {s:.4f}")

# 热图
valid = [c for c in characters if c in model_sg.wv]
mat = np.array([[model_sg.wv.similarity(a, b) for b in valid] for a in valid])
plt.figure(figsize=(8, 7))
sns.heatmap(mat, annot=True, cmap="YlOrRd",
            xticklabels=valid, yticklabels=valid, fmt=".3f")
plt.title("《三国演义》主要人物亲密度热图")
plt.tight_layout()
plt.savefig("result/sanguo_heatmap.png", dpi=150)
plt.show()
```

与【刘备】最相关的5个词：

久：0.9815
主公：0.9760
何不：0.9755
言正合：0.9738
当：0.9733

与【张飞】最相关的5个词：

赵云：0.9899
魏延：0.9836
黄忠：0.9793
引兵：0.9733
夏侯惇：0.9711

与【关羽】最相关的5个词：

许：0.9963
翼：0.9960
谡：0.9950
魏兵屯：0.9942
何能：0.9942

与【曹操】最相关的5个词：

孙权：0.9584
许都：0.9317
闻孙策：0.9297

与【曹操】最相关的5个词：

孙权：0.9584
许都：0.9317
闻孙策：0.9297
成都：0.9293
郭二贼：0.9289

与【孙权】最相关的5个词：

鲁肃：0.9870
董卓：0.9773
使者：0.9769
书：0.9767
献计：0.9741

与【诸葛亮】最相关的5个词：

必为：0.9957
大王：0.9956
掌握：0.9955
神人：0.9953
奈何：0.9948

六人两两亲密度：

刘备 ↔ 张飞：0.2917
刘备 ↔ 曹操：0.6759
刘备 ↔ 孙权：0.7460
刘备 ↔ 诸葛亮：0.9639
张飞 ↔ 曹操：0.6278
张飞 ↔ 诸葛亮：0.4541

与【诸葛亮】最相关的5个词：

必为：0.9957

大王：0.9956

掌握：0.9955

神人：0.9953

奈何：0.9948

六人两两亲密度：

刘备 ⇔ 张飞：0.2917

刘备 ⇔ 曹操：0.6759

刘备 ⇔ 孙权：0.7460

刘备 ⇔ 诸葛亮：0.9639

张飞 ⇔ 曹操：0.6278

张飞 ⇔ 诸葛亮：0.4541

关羽 ⇔ 刘备：0.7783

关羽 ⇔ 张飞：0.7831

关羽 ⇔ 曹操：0.8051

关羽 ⇔ 孙权：0.8738

关羽 ⇔ 诸葛亮：0.8850

曹操 ⇔ 诸葛亮：0.6726

孙权 ⇔ 张飞：0.5987

孙权 ⇔ 曹操：0.9584

孙权 ⇔ 诸葛亮：0.7685

热图已保存

(nlp311) C:\Users\lenovo\Desktop\实验5>

